Token-Tracker Harness — Summary

A harness that governs AI coding agents through declared guardrails, structured checkpoints, clean material handling, and named alarms — operating in the domain of token-budget governance.

Architecture

Single FastAPI process serving three roles:

Role	What it does
LLM Proxy	Agent calls arrive at the harness. LiteLLM forwards to real provider. Governance injected before forwarding.
REST API	Session management, guardrail CRUD, checkpoint evaluation, alarm history, AGILE board.
Portal UI	HTMX + Jinja2. Dashboard with token charts, AGILE board, session history, alarm log.

Agent-agnostic — Hermes, Claude Code, Codex all work. LiteLLM normalizes 100+ providers. Single Docker container, SQLite, local or cloud.

Four Pillars

Guardrails (6 declared in `guardrails.yaml`)

#	Guardrail	Enforcement
G1	Daily token cap (user-configurable)	Calendar-day reset at midnight. Notify when exceeded.
G2	Session token cap	Notify when exceeded.
G3	Budget consumption zones (green/yellow/red)	Three-layer graduated enforcement.
G4	Loop detection (same tool+input N times)	Notify user.
G5	Session timeout	Notify, save checkpoint.
G6	Tool-category budget weighting (no category >50%)	Inject warning context.

Three-layer enforcement per zone:

Layer	Green	Yellow	Red
System prompt	"Be thorough"	"Prioritize, be concise"	"Output only deliverable"
max_tokens	4096	2048	1024

Layer	Green	Yellow	Red
Available tools	All	No web_search/browser	Only read_file, patch, terminal

No hard-stops. Agent is constrained at the transport level — it cannot ignore the limits.

Checkpoints (4 evaluated at session boundaries)

#	Checkpoint	Pass criteria
C1	Budget health	Session spend ≤ fair share of remaining daily budget
C2	Stuck-loop score	<3 loop alarms during session
C3	Tool diversity	≥3 distinct tool categories used
C4	Timeout respect	Session ended before timeout

Stateless evaluator reads session JSON artifacts. Replayable from any checkpoint.

Material Handling

Interface	Direction	Transport
Task submission	User → Harness	AGILE Board
Session dispatch	Harness → Agent	API proxy
Token analytics	Harness → User	Dashboard
Session reports	Harness → User	Portal + REST API
Guardrail config	User → Harness	YAML + REST API
Alarm notifications	Harness → User	Webhook / macOS notification

Alarms (7 named types)

Alarm	Trigger	Severity
`BUDGET_YELLOW`	Session >60% cap	LOW
`BUDGET_RED`	Session >85% cap	MEDIUM
`DAILY_CAP_EXCEEDED`	Daily cap reached	HIGH
`LOOP_DETECTED`	Same tool+input ≥N times	MEDIUM
`SESSION_TIMEOUT`	Wall-clock exceeded	MEDIUM
`TOOL_CATEGORY_BIAS`	One category >50% budget	LOW
`CHECKPOINT_FAIL`	Checkpoint evaluated as fail	MEDIUM

All structured JSON: type, severity, context, recommended action.

AGILE Board

Columns: **Backlog → Estimated → Running → Review → Done**

- **LLM-assisted estimation** — harness classifies task complexity, estimates tokens, recommends model
- **Model tiers**: Simple→Haiku, Modest→Sonnet, Complex→Opus (budget-aware auto-downgrade when tight)
- **Direct dispatch**: one task → one session → one budget
- **Budget override**: warns if estimate exceeds remaining budget; human can override
- **Midnight reset**: sessions return to green zone; no restart needed
- **Second-worker swap**: re-run any completed task with a different model

Human-in-the-Loop Escalation

The harness constrains the agent, not the human. Six escalation paths:

Path	**Human decision**
Budget override	Override / reduce estimate / cancel
Daily cap exceeded	Raise cap / abort / continue
Loop detected	Continue / abort / adjust threshold
Session timeout	Continue / abort / extend
Checkpoint failure	Review → advance or re-dispatch
Zone escalation	Auto-handled; can override

Tech Stack

Layer	**Technology**
Runtime	Python 3.11+
Framework	FastAPI
Provider abstraction	LiteLLM
Database	SQLite
Portal	HTMX + Jinja2 + Chart.js
Notifications	Webhook + macOS osascript
Container	Docker
Cloud	Fly.io / Railway

Key Design Decisions

- **Soft enforcement**: harness constrains agent behavior via transport-level limits, never hard-

stops

- **Human always has final say**: budget overrides, alarm responses, model selection — all decided by the human
- **Agent-agnostic**: any OpenAI-compatible agent works; LiteLLM normalizes providers
- **Calendar-day budget**: resets at midnight; sessions dynamically re-enter green zone
- **Checkpoint persistence**: JSON artifacts per session; stateless evaluator enables replay
- **Three-layer governance**: system prompt → max_tokens → tool restrictions; agent cannot ignore